RoamPredict

Code architecture and design principles

The layers, the components and how they depend on each other, the data that flows between them, the threads, the database, and every engineering principle the code follows, with the file and line where you can see it.

Companion to “RoamPredict: how the code works”. Code version: commit 2af63f6 · 33 source files, about 7,400 lines in roampredict/, 1,560 in simulator/ · 264 tests · Python 3.11+ standard library only.

Contents

1. Architecture at a glance
2. The layers
3. Components and dependencies
4. Runtime: the control loop and the data flowing through it
5. Sequence of one switch
6. Concurrency: three threads, one decision maker
7. Persistence: the SQLite schema
8. Configuration and composition (how the parts are wired)
9. Design principles, one by one
10. Design patterns used
11. Algorithms and the principle behind each
12. Security principles
13. Testing architecture
14. Package map: every file

1. Architecture at a glance

RoamPredict is a **layered, single-process control loop**. A sensing layer reads Windows' Wi-Fi state; a pure decision core (filter, trend, state machine, ranking) decides; a guarded action layer performs one safe switch; a persistence and observation layer remembers and reports everything. The decision core never talks to Windows directly: it only sees plain Python data objects. That one rule makes the core testable on any OS and lets a simulator and a mock replace Windows.

Property	Choice	Why
Style	Layered architecture + ports and adapters (hexagonal)	Windows-specific code stays at the edge; logic stays portable and testable
Control	Sense → decide → act loop at 4 Hz (closed-loop controller)	Wi-Fi changes within seconds; 0.25 s is fast enough and cheap
Decision model	Finite-state machine fed by filtered signal + trend	Explainable, deterministic, no training data needed
Concurrency	One decision thread; helper threads only produce events into queues	No locks around decisions, no race conditions in the logic
State	SQLite file (memory across runs) + JSON-lines log (audit)	Standard library, zero install, survives restarts
Dependencies	Python standard library only (ctypes, sqlite3, tomllib, socket)	Runs on a fresh Windows laptop with nothing to install
Safety	Dry-run default, safety gate in front of every action, kill switch	The program changes a real network connection

2. The layers

Each layer only calls the layer below it (and the shared data types). Nothing below knows about anything above.

6 Interface
commands for people: parse arguments, load config, wire objects, print reports
`cli.py` `doctor.py` `config.py` `__main__.py`

5 Orchestration
the control loop: owns time, asks the core, calls the action layer, times outages
`engine.py`

4 Action (guarded)
may change the connection, only through the gate
`safety.py` `roam/executor.py`

3 Decision core (pure logic)
no I/O, no clock of its own: same input, same output
`signal/filter.py` `signal/trend.py` `state_machine.py` `topology/predictor.py`

2 Memory and observation
remember and report
`topology/store.py (SQLite)` `telemetry.py` `net/probe.py` `buildinfo.py`

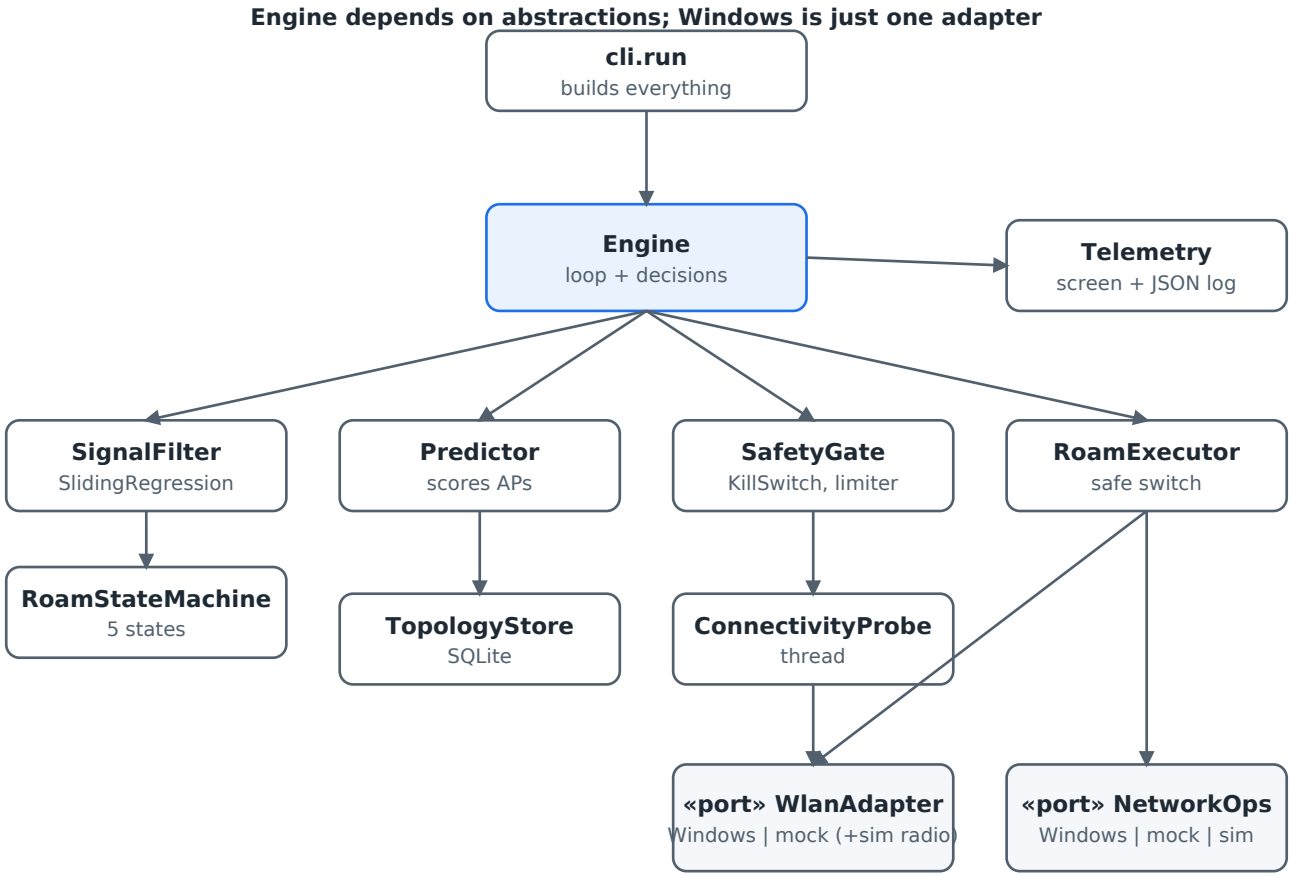
1 Platform ports + adapters
the only code that touches Windows
`platform/base.py` `windows_wlan.py` `win32_structs.py` `info_elements.py` `net/l3.py` `net/latency.py`

0 Shared types and validation
frozen dataclasses, enums, input checks used by every layer
`LinkStatus` `BssEntry` `WlanEvent` `validate.py`

The simulator (simulator/) and the mock (platform/mock.py) plug in at layer 1 instead of Windows, which is why the same engine runs in the virtual school and in the tests.

3. Components and dependencies

Arrows mean “uses”. The engine receives every collaborator from outside (dependency injection); it creates none of them. The two abstract ports, `WlanAdapter` and `NetworkOps`, are implemented for Windows (WindowsWlanAdapter, WindowsNetworkOps) and for tests (MockWlanAdapter, MockNetworkOps); the simulator reuses MockWlanAdapter with its own radio (SimDriver) and SimNetworkOps.



Simplified: the engine also reads the store and the adapter directly, and the probe is owned by the engine (it is drawn under the gate only to save space). No component below the engine imports engine.py.

Component	Responsibility (one each)	Depends on
Engine	Run the loop, combine all results, time outages	every component below, through their public methods
SignalFilter / SlidingRegression	Clean readings / compute slope and time to threshold	nothing (pure)
RoamStateMachine	Turn level + slope into a state, with holds and cooldowns	nothing (pure)
Predictor	Score and filter candidate APs	TopologyStore (read only)
SafetyGate	Say yes or no to an action; trip the kill switch	KillSwitch file, RoamRateLimiter
RoamExecutor	Verify, connect, check IP, fall back, measure	WlanAdapter, NetworkOps, Telemetry
TopologyStore	Persist and query APs, moves, switches, outages	sqlite3
ConnectivityProbe	Detect internet outages in a background thread	a Pinger (TcpPinger)
Telemetry	Write events to screen and JSON log, remove secrets	nothing
WindowsWlanAdapter	Translate wlanapi.dll calls and structs into Python objects	ctypes, win32_structs, info_elements

4. Runtime: the control loop and the data flowing through it

`Engine.run` (engine.py:205) calls `tick()` every 0.25 s on a fixed schedule (if a tick runs late it does not try to catch up). Each stage turns one data type into the next:

Stage	Input	Output (type)	Code
Read events	Windows notifications (queued by callback)	list[WlanEvent]	adapter.poll_events()
Read probe	probe results (queued by thread)	list[ProbeEvent]	probe.poll()
Read link	WlanQueryInterface	LinkStatus (frozen dataclass)	adapter.link_status()
Filter	raw dBm + time	FilteredSample (despiked, smoothed)	SignalFilter.update
Trend	smoothed values of the last 3 s	TrendEstimate (slope, valid)	SlidingRegression.estimate
Decide state	SignalSnapshot (level, slope, time to threshold)	RoamState (enum)	RoamStateMachine.update
Candidates	BSS list from Windows' cache or a scan	list[BssEntry]	adapter.bss_list()
Rank	candidates + learned counts	list[ScoreBreakdown], sorted	Predictor.rank
Verify	best candidate + fresh scan	Verification	RoamExecutor.verify
Gate	link + time	GateDecision(allowed, reason)	SafetyGate.check_roam
Act	link + candidate	RoamOutcome → SwitchRecord	RoamExecutor.switch
Record	outages, switches, associations	SQLite rows + JSON lines	TopologyStore, Telemetry

Principle: **data flows one way, as immutable values**. Every stage returns a new frozen object instead of changing a shared one, so any stage can be tested by giving it an object and checking the object it returns.

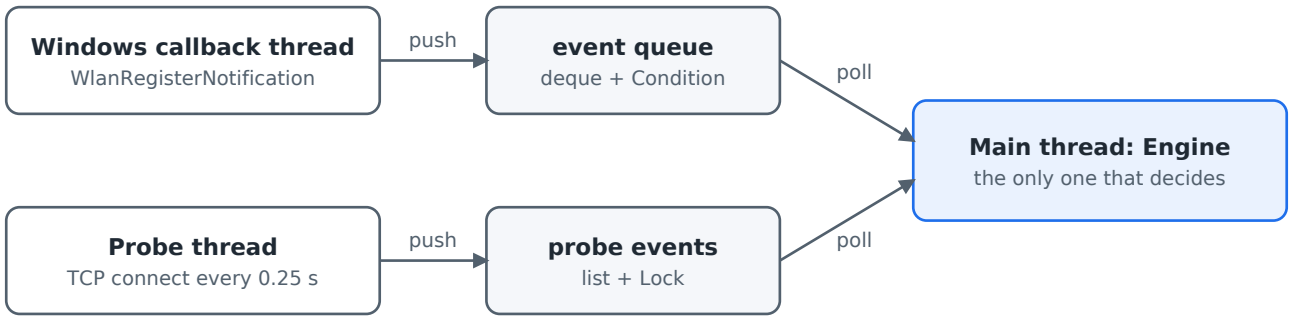
5. Sequence of one switch

What happens between the engine, the gate, the executor and Windows when the state becomes ROAM_IMMINENT:

#	From → to	Call / message	Failure path
1	Engine → Predictor	rank(current, candidates) → sorted ScoreBreakdowns	none eligible → NO_CANDIDATE
2	Engine → Store	transition_counts(): is the usual next AP coming?	wait → WAITING
3	Engine → Executor	verify(): re-read list, scan if older than 3 s, check +8 dB	NOT_VERIFIED

#	From → to	Call / message	Failure path
4	Engine → SafetyGate	check_roam(): dry-run? kill switch? allow-list? rate limit?	WOULD_SWITCH (dry-run) or blocked
5	Engine → StateMachine	begin_roam() → ROAMING	
6	Executor → WlanAdapter	connect_bssid(profile name, ssid, bssid)	802.1X stall → kill switch; other → fallback
7	Windows → Adapter	notifications: associating, authenticating, connected	timeout 12 s
8	Executor → NetworkOps	snapshot(), ARP flush or DHCP renew, gateway check	no IP → hand back to Windows
9	Engine → Gate, SM, Store	record_result(), end_roam() → COOLDOWN, record_switch()	2 failures in a row → kill switch
10	Engine → Telemetry	DECISION, SWITCH_RESULT lines	

6. Concurrency: three threads, one decision maker



This is the **producer–consumer** pattern. Producers never call the engine; they only append to a queue protected by a lock (windows_wlan.py:318, probe.py:134). The engine drains both queues at the start of each tick. So all decisions, all state and all database writes happen on one thread, in one order, and the log has one clock. Telemetry also has a lock (telemetry.py:83) because any thread may log. The probe thread is a daemon and is stopped and joined at the end (probe.stop), and an outage still running is recorded, so stopping never hides one.

7. Persistence: the SQLite schema

One file, `%LOCALAPPDATA%\RoamPredict\roampredict.db`, opened through `TopologyStore` (the only class that writes SQL). WAL mode, parameterised queries only, a schema version in `meta`.

Table	One row is	Main columns	Read by
meta	a setting	key, value (schema_version)	store on open
bss	one access point ever seen	bssid (key), ssid, channel, band, last/max rssi, seen times, 11r/k/v flags	doctor, report

Table	One row is	Main columns	Read by
associations	a connection to an AP	bssid, ssid, at, source (startup / windows / roampredict)	Predictor (“connected recently”)
transitions	a learned move A → B	from_bssid, to_bssid, count, last_at	Predictor (“learned”), WAITING
switch_events	one roam, ours or Windows'	initiator, from/to/requested bssid, success, total/assoc/auth/dhcp ms, dry_run, detail (JSON)	report (switch times)
outages	one outage	started_at, duration_ms, layer (wifi / internet), mode (dry-run / active), last_rssi	report, --timeline

Principles: **repository pattern** (one class owns the storage; callers get dataclasses such as OutageRecord, never SQL rows), **validation on write** (record_outage rejects an unknown layer or mode and a negative duration), **bounded growth** (old associations are pruned at start), and **learning only from confirmed stays** (a move is counted only after a minimum dwell, and only within one SSID).

8. Configuration and composition

`config.toml` is read with tomllib and checked against `SCHEMA`, a nested dictionary section → key → type (config.py:85). Unknown sections, unknown keys and wrong types are rejected with a clear message instead of being silently ignored. The values are then turned into one frozen dataclass per component (FilterConfig, TrendConfig, StateMachineConfig, PredictorConfig, ExecutorConfig, SafetyConfig, EngineConfig), each of which checks its own ranges in `__post_init__`.

Composition root: `cli.cmd_run` is the one place that creates the real objects and passes them into `Engine(...)` by keyword. Tests and the simulator build the same Engine with different parts:

```
engine = Engine(adapter=WindowsWlanAdapter(...),   # tests: MockWlanAdapter, simulator: SimDriver
                net=WindowsNetworkOps(),          # tests: MockNetworkOps
                store=TopologyStore(db_path),     # tests: TopologyStore(':memory:')
                predictor=Predictor(store, cfg.predictor),
                executor=RoamExecutor(adapter, net, cfg.executor, telemetry),
                safety=SafetyGate(cfg.safety, KillSwitch(path)),
                telemetry=Telemetry(log_path), probe=probe, config=cfg.engine)
```

Defaults are safe defaults: with no config file the program runs, in dry-run, acting on no network.

9. Design principles, one by one

Principle	What it means here	Where to see it
Separation of concerns	Sensing, deciding, acting, remembering and reporting live in different modules	the layer diagram, section 2
Single responsibility	Each class has one reason to change: the filter only filters, the gate only allows or refuses	signal/filter.py, safety.py
Dependency inversion	The engine depends on the abstract ports WlanAdapter and NetworkOps, not on Windows	platform/base.py:198, :316
Dependency injection	Collaborators are passed in, never created inside; clocks are injectable	engine.py:133 (wall_clock=), probe.py (clock=, wall=)
Open/closed	A new backend or smoother is added by a new class, without editing the engine	mock.py, simulator/strategies.py, Kalman1D vs EmaSmoother
Interface segregation	Small protocols: a probe needs only an object with ping() and timeout_s	probe.py:27 _Pinger, latency.py:90 Pinger
Pure core, impure edge	Decision code has no I/O and no global state, so it is deterministic	state_machine.py, trend.py, predictor.rank
Immutability	Data passed between layers are frozen dataclasses	LinkStatus, BssEntry, Outage, ScoreBreakdown
Fail-safe defaults	Dry-run, empty allow-list, kill switch honoured, scan busy treated as fine	safety.py:28, windows_wlan.py:509
Fail loudly at the boundary	ctypes prototypes are declared, so a wrong argument raises instead of corrupting memory	windows_wlan.py:194 PROTOTYPES
Validate all input	BSSIDs, SSIDs, profile names and config are checked before use	validate.py, config.py:_check
Graceful degradation	No dBm from the driver → use quality; no ping → check the address; no admin → skip ARP/DHCP	windows_wlan.py:449, executor.py _check_l3
Hand control back	On any doubt (login prompt, no IP), stop acting and let Windows decide	executor.py switch() fallbacks
Hysteresis and debouncing	Different enter/exit thresholds and hold times prevent flapping	state_machine.py _Hold, -67 / -63 dBm
Measure, don't assume	Every switch and outage is timed and stored; our own switches count against us	engine._switch, store.outages
Observability	One event stream: short screen lines + full JSON log with a version stamp	telemetry.py, buildinfo.py
Configuration as data	All thresholds in dataclasses with defaults, overridable by a validated TOML file	config.py SCHEMA
Zero dependencies	Standard library only; nothing to install on the target laptop	pyproject.toml dependencies = []
Testability first	Every part runs on Linux with fakes and simulated time; 264 tests	tests/, platform/mock.py
Honesty in the output	Every “no” has a reason in the log; limits are written in the README	NO_CANDIDATE … closest: …

10. Design patterns used

Pattern	Where	Role in RoamPredict
Adapter / Ports and adapters	WindowsWlanAdapter, MockWlanAdapter (+ SimDriver radio), SimNetworkOps	Same interface for real Windows, tests and the simulator
Abstract base class (template)	WlanAdapter(abc.ABC), NetworkOps(abc.ABC)	Defines the contract; default methods like reason_text
State machine (state pattern)	RoamStateMachine + RoamState enum	Behaviour depends on STABLE / DEGRADING / ROAM_IMMINENT / ROAMING / COOLDOWN
Strategy	smoother = "ema" | "kalman"; simulator strategies	Swap an algorithm without changing callers
Observer / callback	WlanRegisterNotification → _on_notification; on_transition	Windows and the state machine notify listeners
Producer–consumer queue	event deque, probe event list	Threads hand data to the single decision thread
Repository	TopologyStore	Hides SQL; returns dataclasses
Composition root + DI	cli.cmd_run → Engine(...)	Wiring in one place
Facade	Engine for the CLI; RoamExecutor.switch for one switch	One call hides many steps
Guard / gatekeeper	SafetyGate.check_roam	One checkpoint in front of every action
Circuit breaker	KillSwitch tripped after 2 failures or a login prompt	Stops repeated harm until a person resets it (unkill)
Rate limiter (sliding window)	RoamRateLimiter (deque of timestamps)	≥ 20 s between switches, ≤ 6 per 10 min
Value object	frozen dataclasses everywhere	Safe to share, easy to compare in tests
Command pattern (CLI)	argparse sub-commands, each with set_defaults(func=...)	doctor, scan, run, report, simulate, latency, kill, unkill, forget, paths

11. Algorithms and the principle behind each

Algorithm	Where	Principle
Median + MAD outlier rejection with confirmation	SpikeRejector (filter.py:62)	Robust statistics: a single bad reading cannot move the median
Exponential moving average with time-based factor	EmaSmoother (filter.py:117)	Low-pass filter that works with uneven sample times
1-D Kalman filter (option)	Kalman1D (filter.py:142)	Optimal estimate from a noise model
Least-squares line + t-statistic	linear_fit, SlidingRegression (trend.py)	Only trust a slope that is clearly larger than its error
Time to threshold	time_to_threshold (trend.py:146)	Predict instead of react: act before the level is reached
Finite-state machine with dwell timers	RoamStateMachine (state_machine.py)	Hysteresis and debouncing against flapping
Weighted multi-criteria score	Predictor.rank (predictor.py:158)	Combine learned, signal, freshness and recency on one 0–1 scale
Laplace-smoothed transition probability	learned_p in rank	A move seen once is not certain; unseen moves are not impossible
Exponential decay of freshness	seen term, tau 20 s	Old observations count less
Hard eligibility margin (+8 dB, +3 dB dead link)	rank, verify	Avoid ping-pong; act only on a clear improvement
Sliding-window rate limit	RoamRateLimiter (safety.py:90)	Bound the damage of a wrong decision
Consecutive-loss outage detection	OutageTracker (probe.py:58)	Two lost probes, not one, so packet loss is not an outage
Median / p90 statistics	store.switch_stats, outage_stats	Robust summaries for small samples

12. Security principles

- **Least privilege**: runs as a normal user; admin is optional and only adds ARP flush and DHCP renew.
- **No secrets handled**: WlanConnect receives only the saved profile's name; credentials stay in Windows. telemetry.scrub (telemetry.py:51) replaces any log field whose key looks like a password, token or certificate; buildinfo reads .git/HEAD and refs but never .git/config (which can contain tokens).
- **Input validation at the trust boundary**: every BSSID is normalised and must be unicast (validate.py:50); SSIDs ≤ 32 bytes; profile names ≤ 255 characters; config keys and types checked.
- **No shell**: subprocess calls use absolute paths and argument lists, never shell=True; no user text is put into a command line.
- **Scope limits**: own device, allow-listed SSIDs, optional allow-listed profiles, deny-listed BSSIDs.

- **Defence in depth**: dry-run default + allow-list + kill switch + rate limit + failure breaker + login-prompt stop; any one of them alone prevents an unwanted switch.
- **Fail closed for actions, fail open for observation**: if anything is unclear the program does not switch, but it keeps measuring and logging.

13. Testing architecture

Level	How	Examples
Unit (pure logic)	Feed values, check results; no Windows needed	test_filter, test_trend, test_state_machine, test_predictor
Contract / structure	ctypes structures checked against Windows x64 sizes and offsets	test_win32_structs (30 tests)
Adapter with a fake DLL	A stand-in for wlanapi.dll built from the same PROTOTYPES	test_windows_adapter, tests/wlan_fakes.py
Component with mocks	MockWlanAdapter + MockNetworkOps with simulated time	test_executor, test_engine (30 tests)
Persistence	TopologyStore(':memory:')	test_store
System simulation	Virtual school floor, walkers, radio model, 3 strategies	test_simulator, roampredict simulate
Field tests	Real walks with --probe, dry-run vs active, report --timeline	walk logs in the README

Principles: **simulated time** (the mock's clock advances only when the code sleeps, so a 60-second scenario runs in milliseconds), **regression tests from field bugs** (each real failure, such as the frozen reading or the missing IP after a switch, became a test before the fix), and **the same code path in tests and in production** (only the adapters differ).

14. Package map: every file

File	Lines	Layer	Contains
cli.py	519	6	argparse commands, composition root, report and timeline
doctor.py	306	6	read-only health checks
config.py	305	6	TOML loading, SCHEMA validation, building config dataclasses
engine.py	936	5	Engine, EngineConfig: loop, decisions, outage timing
safety.py	167	4	SafetyConfig, KillSwitch, RoamRateLimiter, SafetyGate
roam/executor.py	463	4	ExecutorConfig, Verification, RoamOutcome, RoamExecutor
signal/filter.py	216	3	SpikeRejector, EmaSmoother, Kalman1D, SignalFilter
signal/trend.py	158	3	linear_fit, SlidingRegression, time_to_threshold
state_machine.py	252	3	RoamState, RoamStateMachine, Transition
topology/predictor.py	260	3	PredictorConfig, ScoreBreakdown, Predictor
topology/store.py	570	2	TopologyStore, records and stats dataclasses, schema
telemetry.py	114	2	Telemetry, scrub
net/probe.py	177	2	OutageTracker, ConnectivityProbe
buildinfo.py	72	2	git commit and folder for --version
platform/base.py	332	1/0	ports WlanAdapter, NetworkOps; LinkStatus, BssEntry, WlanEvent, L3Snapshot
platform/windows_wlan.py	649	1	WindowsWlanAdapter over wlanapi.dll
platform/win32_structs.py	451	1	ctypes copies of the Windows structures
platform/info_elements.py	193	1	beacon parsing: 802.11r/k/v, RSN
platform/mock.py	440	1 (test)	scripted Wi-Fi and IP stack
net/l3.py	241	1	IP address, ping, ARP flush, DHCP renew
net/latency.py	371	1	TcpPinger, IcmpPinger, latency and bufferbloat test
validate.py	114	0	BSSID, SSID and profile-name checks
simulator/ (7 files)	1,559	1 (sim)	floor plan, radio, walker, link model, strategies, run, plot